

Programação Funcional em Haskell

Avaliação e concorrência

REALIZAR APÓS posição 53: 14.3 Concorrência - Async

Posições na playlist 45 a 53 | 120 minutos | 100 pontos

Curso principal: Programação Funcional em Haskell - Curso Completo - UFABC |
<https://www.youtube.com/watch?v=eTisiy5FB7k&list=PLYItvall0TqJ25sVTLcMhxsE0Hci58mpQ&index=1>

Conteúdo verificável: Laziness; listas infinitas; Eval; estratégias; Threadscope; forkIO; MVar; Async

Instruções

Resolva sem consultar o gabarito. Apresente desenvolvimento, hipóteses, unidades e verificações. Código pode ser escrito em papel ou editor sem sugestões automáticas; use somente recursos compatíveis com o bloco. Ferramentas de simulação são permitidas apenas onde solicitadas. Use folhas adicionais para diagramas e cálculos.

Pontuação: Q1 = 15; Q2 = 25; Q3 = 25; Q4 = 35. Crédito parcial depende de raciocínio demonstrado. Meta: ≥ 80 prosseguir; 70–79 revisar erros; < 70 revisar o bloco.

Aluno(a): _____ Data: _____

Alinhamento: títulos/ordem e informações públicas do curso. As aulas não foram integralmente assistidas; detalhes não expostos não estão certificados.

Laziness

- (a) Explique por que take5 [1..] termina. (b) Defina sequência de Fibonacci de forma preguiçosa. (c) Discuta consumo de memória quando referências a prefixos já usados são mantidas.

Desenvolvimento / justificativa / esquema:

Paralelismo e avaliação

- (a) Explique como separar duas computações puras para avaliação paralela com estratégias/Eval.
(b) Distingua produzir um thunk de forçar resultado suficientemente. (c) Proponha medição com Threadscope e controle de overhead.

Desenvolvimento / justificativa / esquema:

Threads e MVar

Projete comunicação de um resultado entre thread trabalhadora e principal usando MVar inicialmente vazio. (a) Dê sequência `forkIO`, `putMVar`, `takeMVar`. (b) Explique bloqueio. (c) Identifique cenário de deadlock por perda do produtor.

Desenvolvimento / justificativa / esquema:

Async e conclusão

(a) Compare lançar thread solta com usar abstração Async para esperar resultado. (b) Discuta propagação de exceções e cancelamento. (c) Proponha desenho para duas tarefas em que falha de uma deve encerrar trabalho da outra, sem depender de espera fixa por tempo.

Desenvolvimento / justificativa / esquema:

Gabarito e Resoluções

Abra esta seção somente depois de concluir a tentativa. Há mais de um método correto possível; compare também o processo e os casos extremos.

Critérios de correção

Q1: 4 pontos pela modelagem/conceito, 8 pela solução, 3 pela verificação. Q2 e Q3: 6 pela modelagem, 14 pela solução, 5 pela justificativa/verificação. Q4: 8 pela modelagem, 20 pela solução, 7 pela análise crítica e casos extremos. Nos itens dissertativos, a solução inclui os conceitos e relações obrigatórios; crédito integral exige conectar esses conceitos ao cenário. Em código, valorize correção do contrato, casos base e explicação; uma chamada a biblioteca que evita a implementação pedida não substitui o algoritmo.

Questão 1 - Laziness (15 pontos)

take exige somente prefixo finito, não materializa toda lista. Definição: `fibs=0:1:zipWith (+) fibs(tail fibs); take7` fornece `[0, 1, 1, 2, 3, 5, 8]`. Manter referência à cabeça pode impedir coleta dos prefixos; preguiça não garante memória constante. Também não torna toda definição infinita produtiva: é preciso conseguir produzir cada próximo elemento.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 2 - Paralelismo e avaliação (25 pontos)

Construir tarefas independentes, usar estratégia de avaliação apropriada e sincronizar resultados; grau de forçamento importa, pois WHNF pode deixar estrutura interna pendente. Threadscope ajuda observar atividade de capacidades, sparks e gargalos. Comparar mesma tarefa, resultado e configurações, com tamanhos que amortizem criação de tarefas. Paralelizar operações pequenas pode piorar tempo. Não prometer speedup só pela presença de `rpar`/estratégia.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 3 - Threads e MVar (25 pontos)

Criar MVar vazio; `forkIO` calcula resultado e `putMVar`; principal `takeMVar` e prossegue com resultado. `take` em vazio bloqueia; `put` em cheio bloqueia segundo contrato. Se produtor termina com falha antes de `put`, consumidor pode esperar para sempre. Gestão de falhas e ciclo de vida é necessária; MVar não detecta automaticamente desaparecimento do produtor.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Questão 4 - Async e conclusão (35 pontos)

Async fornece `handle` e operações de espera/cancelamento; abstração estruturada liga vida da tarefa ao escopo e torna falha observável. Esperar conclusão real, propagar falha e cancelar tarefa irmã quando política exige. Uma espera de tempo fixo não prova conclusão e pode deixar trabalho órfão. A resposta deve declarar política de cancelamento e limpeza; detalhes de API precisam corresponder à versão usada no curso, sem exigir uma função não apresentada.

Aceitar outra solução que preserve o contrato e demonstre o resultado. Se um erro local não impedir os passos seguintes, avaliar esses passos independentemente.

Diagnóstico do checkpoint

Registre sua nota e os conceitos a revisar em progresso.csv. Antes de avançar, refaça os itens errados sem olhar a resolução e crie um pequeno exemplo novo que use a mesma ideia. Nota alta sem justificar respostas não demonstra domínio.